SCICALC PRO

Documentación Técnica Integral

PRD • ERS (IEEE 830) • RFC • Design Specs

Versión: 5.0
Fecha: 09 de Marzo de 2026
Autores: Puentes Angel, Bonilla Santiago, Patiño Kevin

ODS 4: Educación de Calidad
Arquitectura Hexagonal • Web-First • Alta Precisión

Stack: HTML/CSS/JS (Jinja2) • Python/FastAPI • PostgreSQL

Índice General

- DOCUMENTO 1: PRD (Product Requirements Document)
  - 1.1 Visión del Producto
  - 1.2 Problema y Justificación
  - 1.3 KPIs Técnicos Detallados
  - 1.4 Usuarios Objetivo
  - 1.5 Roadmap del Producto
- DOCUMENTO 2: ERS – Especificación de Requisitos de Software (IEEE 830)
  - 2.1 Introducción
  - 2.2 Descripción General del Sistema
  - 2.3 Requerimientos Funcionales del Motor Matemático
  - 2.4 CRUDs Obligatorios del Backend
  - 2.5 Requerimientos No Funcionales
  - 2.6 Requerimientos de Seguridad (ISO 27001)
  - 2.7 Requerimientos de Soporte (ITIL)
- DOCUMENTO 3: Design Specs – Especificaciones de Diseño
  - 3.1 Arquitectura Hexagonal: Relaciones de Objetos
  - 3.2 Estructura de Carpetas
  - 3.3 Esquema de Tablas – PostgreSQL
  - 3.4 Gestión de Sesiones Web
  - 3.5 Integración de mpmath en Flujo Asíncrono
  - 3.6 Validación con Expresiones Regulares (Catálogo)
  - 3.7 UI/UX: Especificaciones de Diseño Visual
  - 3.8 Gestión de Riesgos
  - 3.9 Plan de Pruebas y Métricas de Calidad (ISO/IEC 25010)
  - 3.10 Recomendaciones de Control (COBIT)

DOCUMENTO 1: PRD (Product Requirements Document)

1.1 Visión del Producto

SciCalc Pro es un **prototipo web** de calculadora científica de alta precisión diseñado bajo el paradigma **Web-First**. El producto está dirigido a ingenieros, científicos, estudiantes de STEM y profesionales que requieren una herramienta de cálculo confiable accesible desde cualquier navegador moderno, sin necesidad de instalación. La aplicación se alinea con el ODS 4 (Educación de Calidad) al proveer una herramienta educativa gratuita y accesible que democratiza el acceso a cálculos científicos avanzados.

La estrategia Web-First implica que todas las funcionalidades de cálculo, conversión de unidades, historial y memoria son accesibles desde el navegador con persistencia centralizada en PostgreSQL en el servidor. La autenticación JWT garantiza que el historial y las preferencias del usuario estén disponibles en cualquier dispositivo con acceso al navegador.

1.2 Problema y Justificación

Los estudiantes de ingeniería y ciencias frecuentemente requieren herramientas de cálculo científico avanzado en entornos universitarios, laboratorios o de trabajo donde las herramientas disponibles son limitadas o no ofrecen precisión suficiente. Las calculadoras científicas físicas carecen de historial persistente, acceso multiplataforma y capacidad de respaldo. Las aplicaciones existentes en el mercado requieren instalación o no ofrecen precisión arbitraria. SciCalc Pro resuelve esta brecha ofreciendo precisión de punto flotante arbitraria (mediante mpmath) en un prototipo web accesible desde cualquier navegador moderno, con historial centralizado en el servidor.

1.3 KPIs Técnicos Detallados

KPI	Métrica Objetivo	Herramienta de Medición
Precisión de punto flotante	Tasa de error relativo < 10^{-10}	Suite de pruebas mpmath vs. Wolfram Alpha
Latencia P95 de cálculo	< 50ms para operaciones estándar	k6 / Artillery + métricas internas
Latencia P95 de carga de historial	< 500ms	JMeter / Lighthouse
Cobertura de código (Backend)	> 90% (Líneas y Branches)	pytest-cov + SonarQube
Cobertura de código (Frontend)	> 80%	Jest + SonarQube
Rating SonarQube	'A' en Reliability, Security, Maintainability	SonarQube Server
Tamaño del bundle web	< 5 MB (activos estáticos comprimidos)	Lighthouse / DevTools
Tiempo de carga inicial (LCP)	< 2s bajo conexión estándar	Lighthouse / PageSpeed Insights
Tasa de errores HTTP	< 1% bajo carga normal	k6 + logs del servidor
MTTR (Mean Time To Recovery)	< 4 horas	Jira + monitoreo de incidentes

1.4 Usuarios Objetivo

Persona	Descripción	Necesidad Principal
Estudiante de Ingeniería	Estudia cálculo, física, álgebra lineal en entornos universitarios	Funciones trigonométricas, notación científica, precisión, acceso desde cualquier equipo del campus
Investigador Científico	Requiere precisión arbitraria	mpmath con > 10 decimales, funciones especiales
Ingeniero en Oficina	Trabaja en proyectos técnicos con acceso al navegador	Cálculos con historial persistente, acceso multiplataforma desde cualquier navegador
Contador/Financiero	Cálculos rápidos con memoria	Historial auditable, funciones de memoria M+/M-

1.5 Roadmap del Producto

1.5.1 MVP (Meses 1–3)

Motor matemático con operaciones aritméticas básicas y funciones trigonométricas. Persistencia centralizada en PostgreSQL. Interfaz web básica renderizada con Jinja2 en modo estándar y científico. Autenticación JWT para el backend FastAPI. Historial de las últimas 50 operaciones por usuario. 7 pruebas unitarias mínimas en el backend. Análisis estático con SonarQube.

1.5.2 Fase 2: Expansión (Meses 4–6)

Conversor de unidades completo (Longitud, Masa, Temperatura, Volumen, Velocidad). Funciones de memoria (M+, M−, MR, MC). Edición de expresiones con cursor. Modo Oscuro/Claro adaptativo. Notación científica automática.

Pruebas E2E automatizadas con Selenium. Pruebas de rendimiento con k6 y Lighthouse.

1.5.3 Fase 3: Escalado (Meses 7–12)

Precisión arbitraria vía mpmath expuesta como microservicio. Funciones avanzadas: matrices, integrales numéricas, raíces de polinomios. Exportación de historial a .json/.csv. Ayuda y FAQ integrada en el prototipo web. Despliegue en producción con Nginx + Gunicorn + HTTPS. Cobertura de código > 90%.

DOCUMENTO 2: ERS – Especificación de Requisitos de Software (IEEE 830)

2.1 Introducción

2.1.1 Propósito

Este documento especifica los requisitos funcionales y no funcionales del sistema SciCalc Pro conforme al estándar IEEE 830-1998. Está dirigido al equipo de desarrollo, al equipo de QA, y a los stakeholders del proyecto. Sirve como contrato técnico para la implementación.

2.1.2 Alcance

SciCalc Pro es un **prototipo web** con backend en Python/FastAPI que provee cálculos matemáticos de alta precisión accesibles desde el navegador. La persistencia centralizada en PostgreSQL garantiza que el historial y las preferencias del usuario estén disponibles en cualquier sesión autenticada. La arquitectura sigue el patrón Hexagonal (Puertos y Adaptadores) para máxima testabilidad.

2.1.3 Definiciones y Acrónimos

Término	Definición
JWT	JSON Web Token – Estándar de autenticación stateless
CRUD	Create, Read, Update, Delete – Operaciones básicas de persistencia
mpmath	Librería Python de aritmética de punto flotante con precisión arbitraria
DEG/RAD	Grados/Radianes – Modos de entrada angular
P95	Percentil 95 – Métrica de latencia donde el 95% de las peticiones están por debajo
AES-256	Advanced Encryption Standard con clave de 256 bits
Regex	Expresiones Regulares – Patrones de validación de entrada de datos
LCP	Largest Contentful Paint – Métrica de rendimiento de carga web
SPA	Single Page Application – Aplicación web de página única

2.2 Descripción General del Sistema

2.2.1 Perspectiva del Producto

SciCalc Pro opera como un prototipo web accesible desde el navegador, con un backend FastAPI que gestiona la autenticación JWT, los CRUDs de persistencia en PostgreSQL y la lógica de cálculo científico. La interfaz web se renderiza mediante Jinja2 y se comunica con el backend a través de la API REST autenticada.

2.2.2 Restricciones de Diseño

La arquitectura debe seguir estrictamente el patrón Hexagonal. La lógica de dominio (motor matemático, parser, lexer) no puede tener dependencias directas de frameworks, bases de datos o interfaces de usuario. Toda comunicación entre capas se realiza mediante Puertos (interfaces abstractas) e implementaciones concretas (Adaptadores). Todas las entradas de datos del usuario deben ser validadas mediante expresiones regulares antes de procesarse.

2.3 Requerimientos Funcionales del Motor Matemático

RF-001: Operaciones Aritméticas Básicas

Código: RF-001
Actor: Usuario General

Descripción IEEE 830: El sistema deberá permitir al usuario realizar operaciones de suma, resta, multiplicación y división con números enteros y decimales. El sistema debe validar la entrada mediante Regex (patrón: `^-?\\d+(\\.\\d+)?([eE][+-]?\\d+)?$`) para prevenir inyección de caracteres inválidos. La división por cero debe retornar un objeto de error tipado, no una excepción no controlada.

Criterios de Aceptación: El resultado se muestra en < 50ms (P95). La división por cero muestra "Error: División por cero". La validación Regex rechaza entradas como "abc", "12..5", o "" (vacío).
Historia de Usuario: Como estudiante, quiero realizar sumas y restas rápidas validadas para resolver problemas cotidianos con confianza en la entrada.

RF-002: Funciones Trigonométricas

Código: RF-002
Actor: Usuario General / Estudiante de Ingeniería
Descripción IEEE 830: El sistema deberá proveer funciones sin, cos, tan y sus inversas (asin, acos, atan). La entrada angular se valida con Regex según el modo: DEG (`^-?\\d+(\\.\\d+)?$` rango 0–360) o RAD (`^-?\\d+(\\.\\d+)?$` rango 0–2π). El motor matemático interno utiliza mpmath con precisión configurable (mínimo 15 dígitos decimales) para garantizar que sin(90°) = 1.0 exacto.

Criterios de Aceptación: sin(90) en modo DEG retorna 1.0000000000 (10 decimales). El usuario alterna DEG/RAD mediante un toggle visible. tan(90°) muestra "Indefinido".
Historia de Usuario: Como ingeniero, quiero calcular el coseno de un ángulo en radianes con precisión arbitraria para mis cálculos de física.

RF-003: Historial de Operaciones

Código: RF-003
Actor: Usuario General
Descripción IEEE 830: El sistema deberá registrar y almacenar las últimas 50 operaciones en PostgreSQL asociadas a la cuenta del usuario. Cada registro incluye: id (UUID v4), expression (string), result (string), precision_digits (int), created_at (ISO 8601 timestamp), is_deleted (boolean para borrado lógico). El historial es recuperable y cada ítem puede reutilizarse en una nueva operación.

Criterios de Aceptación: Al hacer scroll se muestra la lista paginada. Al hacer clic en un ítem, el valor se copia al campo de entrada. El borrado es lógico (is_deleted = true), no físico.
Historia de Usuario: Como ingeniero, quiero ver el historial de mis cálculos anteriores para auditar o reutilizar resultados.

RF-004: Funciones de Memoria (M+, M–, MR, MC)

Código: RF-004
Actor: Usuario General
Descripción IEEE 830: El sistema deberá disponer de funciones M+, M–, MR y MC. El valor en memoria persiste durante la sesión activa del navegador. La memoria se valida con Regex antes de almacenar para prevenir corrupción de datos.

Criterios de Aceptación: Guardar un número muestra indicador "M". MR recupera el valor exacto almacenado en la precisión configurada.

RF-005: Conversor de Unidades

Código: RF-005
Actor: Usuario General
Descripción IEEE 830: El sistema deberá incluir un módulo de conversión entre unidades de Longitud, Masa, Temperatura, Volumen y Velocidad. La conversión se realiza en tiempo real con validación Regex de la entrada numérica. Los factores de conversión están definidos como constantes inmutables en el dominio.

Criterios de Aceptación: 1000 metros → 1 km. 0°C → 32°F. Conversión en < 50ms.

RF-006: Edición de Expresiones con Cursor

Código: RF-006
Actor: Usuario General
Descripción IEEE 830: El sistema deberá permitir al usuario posicionar un cursor dentro de la expresión matemática para editar sin borrar toda la operación. El parser/lexer del motor matemático debe re-evaluar la expresión completa tras cada edición.

Criterios de Aceptación: El cursor es visible y parpadea. Se puede cambiar "150+20" a "150-20" haciendo clic entre el 0 y el +.

RF-007: Modo Oscuro/Claro

Código: RF-007
Descripción IEEE 830: El sistema deberá adaptar su interfaz según la preferencia del navegador (`prefers-color-scheme`) o selección manual del usuario. El contraste debe cumplir WCAG 2.1 AA (ratio mínimo 4.5:1 para texto normal).

RF-008: Notación Científica

Código: RF-008
Descripción IEEE 830: El sistema deberá formatear automáticamente resultados extremadamente grandes ($> 10^{12}$) o pequeños ($< 10^{-12}$) en notación científica. El usuario puede ingresar números con la tecla EXP. La validación Regex del formato de entrada científica es: `^-?\\d+(\\.\\d+)?[eE][+-]?\\d+$`

2.4 CRUDs Obligatorios del Backend (FastAPI + PostgreSQL)

El backend expone al menos 7 CRUDs completos con borrado lógico (campo is_deleted: boolean). Todos los endpoints requieren autenticación JWT válida. Todas las entradas se validan con Regex y Pydantic validators.

#	CRUD	Entidad	Campos Principales	Reglas de Negocio
1	Usuarios	User	id, email, password_hash, display_name, created_at, is_deleted	Email validado con Regex RFC 5322. Password mínimo 8 caracteres, 1 mayúscula, 1 número.
2	Perfiles	Profile	id, user_id (FK), avatar_url, preferred_precision, angle_mode, theme, is_deleted	preferred_precision entre 2 y 50 decimales. angle_mode: DEG|RAD.
3	Historial	CalculationHistory	id (UUID), user_id (FK), expression, result, precision_digits, created_at, is_deleted	Máximo 50 registros activos por usuario. FIFO automático.
4	Preferencias	UserPreference	id, user_id (FK), key, value, updated_at, is_deleted	key validada con Regex `^[a-zA-Z_]{3,50}$`. value máximo 500 caracteres.
5	Conversiones Favoritas	FavoriteConversion	id, user_id (FK), from_unit, to_unit, category, is_deleted	category: LENGTH|MASS|TEMP|VOLUME|SPEED.
6	Sesiones	Session	id, user_id (FK), browser_agent, jwt_token_hash, expires_at, is_active, is_deleted	Expiración automática. Máximo 5 sesiones activas por usuario.
7	Registros de Auditoría	AuditLog	id, user_id (FK), entity_type, entity_id, action, status, timestamp, is_deleted	Registro de cada operación crítica de la API para trazabilidad.

2.5 Requerimientos No Funcionales

RNF-001: Rendimiento (Latencia)

El sistema deberá mostrar el resultado de cualquier operación matemática estándar en un tiempo no mayor a 50ms (P95) después de que el usuario presione el botón de igualdad. Para operaciones de precisión arbitraria (> 15 dígitos), el límite se extiende a 200ms (P95). Las pruebas de rendimiento se ejecutan con k6 y Artillery.

RNF-002: Disponibilidad del Servicio Web

El servicio web debe alcanzar una disponibilidad del 99% en horario de uso académico. Todas las funcionalidades de cálculo, conversión, historial y memoria requieren conexión activa al servidor. El servidor FastAPI debe responder health check (`/health`) en < 100ms.

RNF-003: Usabilidad

Los botones del teclado numérico y operadores básicos ocupan mínimo el 60% del área de la interfaz. Tamaño mínimo de elementos interactivos: 44×44px conforme a WCAG 2.1. Máximo 2 clics para acceder a funciones científicas.

RNF-004: Portabilidad

Compatible con los navegadores modernos: Chrome 90+, Firefox 90+, Safari 14+, Edge 90+. Diseño responsivo que se adapta a pantallas de escritorio, tablet y móvil. El stack HTML/CSS/JS + Jinja2 garantiza compatibilidad multiplataforma sin necesidad de instalación.

RNF-005: Fiabilidad y Precisión

El motor matemático utiliza aritmética de doble precisión (64-bit IEEE 754) por defecto, con la opción de precisión arbitraria vía mpmath. La tasa de error relativo debe ser < 10^{-10} para todas las funciones trigonométricas, logarítmicas y exponenciales. La validación se realiza contra valores de referencia de Wolfram Alpha.

RNF-006: Seguridad

Autenticación JWT con tokens de acceso (15 min TTL) y refresh tokens (7 días TTL). Todas las contraseñas se almacenan como hash bcrypt con salt de 12 rondas. Validación obligatoria con Regex en todas las entradas del backend. Transmisión cifrada mediante HTTPS/TLS en producción.

RNF-007: Mantenibilidad

Arquitectura Hexagonal (Puertos y Adaptadores) con separación estricta entre dominio, aplicación e infraestructura. Cobertura de código > 90% en backend. Rating 'A' en SonarQube para Reliability, Security y Maintainability. Frontend estructurado con separación de plantillas Jinja2, estilos CSS y lógica JavaScript.

RNF-008: Rendimiento del Navegador

Tiempo de carga inicial (LCP) < 2 segundos bajo conexión estándar. Score de Lighthouse Performance > 80. Uso de CPU del tab del navegador < 5% en reposo. Tamaño total de activos estáticos < 5 MB.

2.6 Requerimientos de Seguridad (ISO 27001)

Código	Requisito	Descripción
RNF-SEC001	Control de Acceso por Credenciales	Autenticación mediante email/contraseña con JWT. Bloqueo de cuenta tras 5 intentos fallidos consecutivos.
RNF-SEC002	Cifrado de Datos en Tránsito	Toda comunicación entre el navegador y el servidor se realiza mediante HTTPS/TLS 1.2+.
RNF-SEC003	Protección contra Inyección	Validación obligatoria con Regex y Pydantic en todas las entradas del backend. Sanitización de salidas en plantillas Jinja2 (auto-escape activo).
RNF-SEC004	Borrado Seguro	Borrado lógico con campo is_deleted. Los registros borrados son inaccesibles para el usuario pero auditables en la base de datos.
RNF-SEC005	Protección de Sesión	Tokens JWT almacenados en sessionStorage. Expiración automática al cerrar el navegador. Cabeceras HTTP de seguridad (X-Frame-Options, X-Content-Type-Options).
RNF-SEC006	Integridad del Servidor	Verificación de integridad de dependencias mediante hash en CI/CD. Escaneo de vulnerabilidades con SonarQube en cada build.

Código	Requisito	Descripción
RNF-SEC007	Gestión de Sesión Inactiva	Expiración del token de acceso tras 15 minutos de inactividad. El refresh token permite renovación hasta 7 días.

2.7 Requerimientos de Soporte (ITIL)

Código	Requisito	Descripción
RNF-SUP001	Registro de Incidentes	Log del servidor con errores críticos, advertencias y excepciones con timestamp ISO 8601.
RNF-SUP002	Exportación de Diagnóstico	Endpoint de health check (`/health`) con estado de conexión a base de datos y métricas básicas.
RNF-SUP003	Ayuda en Línea	Sección FAQ accesible desde la interfaz web con documentación de todas las funciones.
RNF-SUP004	Monitoreo de Capacidad	Alerta si el historial del usuario excede el límite de 50 registros activos.
RNF-SUP005	Exportación de Historial	Funcionalidad para exportar el historial de cálculos a .json/.csv desde la interfaz web.
RNF-SUP006	Notificación de Versiones	Número SemVer visible en la interfaz. Indicador de actualización disponible en el encabezado.
RNF-SUP007	Restablecimiento de Cuenta	Restauración de preferencias predeterminadas desde la interfaz web sin borrar historial (salvo selección explícita).

DOCUMENTO 3: Design Specs – Especificaciones de Diseño

3.1 Arquitectura Hexagonal: Relaciones de Objetos

El flujo de datos sigue la dirección: UI Component → Input Adapter → UseCase → Domain Engine (Parser/Lexer) → Output Adapter. Cada capa se comunica exclusivamente a través de Puertos (interfaces abstractas definidas en el dominio). Los adaptadores implementan estos puertos y son inyectados mediante Dependency Injection.

3.1.1 Flujo Detallado de una Operación de Cálculo

1. UI Component (HTML/CSS/JS): El usuario ingresa la expresión "sin(45)" y presiona "=". El evento `onclick` del botón construye un `CalculationRequest` DTO y lo envía al endpoint de la API via `fetch`.

2. Input Adapter (FastAPI Router): El adaptador de entrada valida la expresión con Regex (`^[0-9a-zA-Z+\\-*/^().sincotaglexpqrt\\s]+$`) para prevenir inyección. Transforma el DTO externo en un comando de dominio `EvaluateExpressionCommand`.

3. UseCase (Application Layer): El caso de uso `EvaluateExpressionUseCase` orquesta la lógica: invoca al Puerto `CalculationEnginePort` con la expresión parseada y al Puerto `HistoryRepositoryPort` para persistir el resultado.

4. Domain Engine – Parser/Lexer: El Lexer tokeniza la expresión en tokens: `[FUNC('sin'), LPAREN, NUMBER(45), RPAREN]`. El Parser construye un AST (Abstract Syntax Tree). El Evaluator recorre el AST y ejecuta el cálculo usando `mpmath.sin(mpmath.radians(45))` si el modo es DEG, o `mpmath.sin(45)` si es RAD. La precisión se configura via `mpmath.mp.dps = precision_digits`.

5. Output Adapter (PostgreSQL): El adaptador de salida implementa `HistoryRepositoryPort` y persiste el resultado en PostgreSQL mediante SQLAlchemy async. Retorna el resultado al router FastAPI que lo envía como JSON al navegador, donde el JavaScript actualiza el DOM.

3.2 Estructura de Carpetas – Arquitectura Hexagonal

```
scicalc-pro/
├── backend/                        (FastAPI)
│   ├── src/
│   │   ├── domain/                 # Núcleo de negocio (Puro)
│   │   │   ├── entities/           # Calculation, User, Profile
│   │   │   ├── value_objects/      # Expression, Precision, AngleMode
│   │   │   ├── ports/              # Interfaces (Input/Output)
│   │   │   └── engine/             # Motor: Lexer, Parser, Evaluator
```

```
│   │   ├── application/            # Casos de Uso (Orquestación)
│   │   │   ├── use_cases/          # EvaluateExpression, ManageHistory
│   │   │   └── dtos/               # Data Transfer Objects
│   │   └── infrastructure/         # Implementaciones Técnicas
│   │       ├── adapters/           # FastAPI Routers, Postgres Repos
│   │       ├── persistence/        # Modelos SQLAlchemy, Migraciones
│   │       └── security/           # JWT, Bcrypt
│   └── tests/                      # Unit, Integration, E2E
├── frontend/                       (HTML/CSS/JS + Jinja2)
│   ├── templates/                  # Plantillas Jinja2
│   │   ├── calculator.html         # Interfaz principal de la calculadora
│   │   ├── login.html              # Página de autenticación
│   │   └── register.html           # Página de registro de usuario
│   └── static/
│       ├── css/                    # Estilos (modo oscuro/claro)
│       └── js/                     # Lógica de interfaz, fetch API
└── docs/                           # PRD, ERS, RFC
```

3.3 Esquema de Tablas – PostgreSQL

```
-- Extensión para IDs seguros
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Usuarios y Autenticación
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) NOT NULL UNIQUE,    -- Regex: ^[a-zA-Z0-9._%+-]+@[...]$
    password_hash VARCHAR(255) NOT NULL,
    display_name VARCHAR(100) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    is_deleted BOOLEAN DEFAULT FALSE
);

-- Historial de Cálculos
CREATE TABLE calculation_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    expression TEXT NOT NULL,              -- Regex: ^[0-9a-zA-Z+\\-*/^().\\s]+$
    result TEXT NOT NULL,
    precision_digits INTEGER DEFAULT 10,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    is_deleted BOOLEAN DEFAULT FALSE
);

-- Registro de Auditoría
CREATE TABLE audit_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    entity_type VARCHAR(50) NOT NULL,
    entity_id UUID NOT NULL,
    action VARCHAR(10) CHECK (action IN ('CREATE','UPDATE','DELETE')),
    status VARCHAR(20) DEFAULT 'success',
    payload JSONB,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

3.4 Gestión de Sesiones Web

Las sesiones de usuario en el prototipo web se gestionan exclusivamente mediante tokens JWT almacenados en `sessionStorage` del navegador. No existe persistencia local de datos de cálculo; toda la información se recupera del servidor PostgreSQL en cada sesión autenticada.

```
# Configuración de tokens JWT en el backend
ACCESS_TOKEN_EXPIRE_MINUTES = 15
REFRESH_TOKEN_EXPIRE_DAYS = 7
ALGORITHM = "HS256"

# La sesión activa registra: user_id, browser_agent, ip_address,
# jwt_token_hash, expires_at, is_active
```

Las tablas de autenticación (`users`, `sessions`) se gestionan exclusivamente en el servidor. La autenticación se realiza mediante credenciales email/contraseña, y el token JWT resultante se almacena en `sessionStorage`, expirando automáticamente al cerrar el navegador.

3.5 Integración de mpmath en Flujo Asíncrono

La librería mpmath es una biblioteca Python BSD de aritmética de punto flotante con precisión arbitraria, actualmente en versión 1.4.0. Provee soporte extensivo para funciones trascendentes, evaluación de sumas, integrales, límites y raíces. La precisión se controla mediante `mp.dps` (decimal places).

Dado que mpmath es CPU-bound y no asíncrono nativamente, la integración con FastAPI (que es async) requiere delegar los cálculos a un `ThreadPoolExecutor` o `ProcessPoolExecutor` para no bloquear el event loop:

```
import asyncio
from concurrent.futures import ProcessPoolExecutor
from mpmath import mp

# Pool de procesos para tareas CPU-bound
executor = ProcessPoolExecutor(max_workers=4)

def _execute_math_logic(expression: str, precision: int) -> str:
    """Ejecución en proceso aislado."""
    mp.dps = precision
    # El motor interno (Parser -> AST -> Evaluator)
    # utiliza mpmath para las funciones trascendentes
    try:
        # Ejemplo simplificado de llamada al motor
        result = motor_evaluador.eval(expression)
        return mp.nstr(result, precision)
    except Exception as e:
        return f"Error: {str(e)}"

async def compute_async(expression: str, precision: int = 15) -> str:
    """Wrapper asíncrono para el servicio de cálculo."""
    loop = asyncio.get_event_loop()
    # Ejecuta el cálculo sin bloquear el servidor FastAPI
    return await loop.run_in_executor(
        executor, _execute_math_logic, expression, precision
    )
```

Para operaciones estándar (precisión <= 15 dígitos), el cálculo se realiza directamente con float nativo de Python (IEEE 754 doble precisión) para maximizar la velocidad (< 50ms P95). Para precisión extendida (> 15 dígitos), se activa mpmath automáticamente, con un límite de latencia extendido a 200ms P95. Esta lógica de selección reside en el UseCase `EvaluateExpressionUseCase`.

3.6 Validación con Expresiones Regulares (Catálogo)

Campo	Patrón Regex	Ejemplo Válido	Ejemplo Inválido
Email	`^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$`	user@mail.com	user@@mail
Password	`^(?=.*[A-Z])(?=.*\\d).{8,}$`	Passw0rd!	pass
Número decimal	`^-?\\d+(\\.\\d+)?$`	-3.14	12..5
Notación científica	`^-?\\d+(\\.\\d+)?[eE][+-]?\\d+$`	1.5e10	1.5e
UUID v4	`^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$`	550e8400-e29b-41d4-a716-446655440000	not-a-uuid
Preference key	`^[a-zA-Z_]{3,50}$`	dark_mode	a
Expresión matemática	`^[0-9a-zA-Z+\\-*/^().\\s]+$`	sin(45)+cos(30)	rm -rf /
Unidad de medida	`^[a-zA-Z°/]{1,20}$`	km/h	

3.7 UI/UX: Especificaciones de Diseño Visual

3.7.1 Elementos Interactivos

Todos los botones interactivos tienen un tamaño mínimo de 44×44px conforme a las guías de accesibilidad WCAG 2.1 y las recomendaciones de Google para interfaces web. El área de clic efectiva puede exceder el área visual del botón para mejorar la accesibilidad. El espacio mínimo entre elementos adyacentes es de 8px.

3.7.2 Distribución de Pantalla

Área	% de Pantalla	Descripción
Display de resultado	15–20%	Muestra expresión actual y resultado con scroll horizontal
Teclado numérico + operadores	60–65%	Botones grandes para minimizar errores de clic
Barra de funciones científicas	15–20%	Accesible con máximo 2 clics. Toggle DEG/RAD visible
Barra de estado/navegación	5%	Indicador de memoria (M), modo, estado de sesión

3.7.3 Adaptabilidad de Pantalla

La interfaz se adapta a diferentes tamaños de pantalla mediante CSS responsive con breakpoints estándar. En pantallas anchas (> 768px), el layout puede mostrar el teclado y el historial en paralelo. En pantallas pequeñas (< 480px), las funciones científicas se acceden mediante un panel desplegable. En ventanas anchas se activa automáticamente el modo científico extendido con más funciones visibles.

3.8 Gestión de Riesgos

ID	Riesgo	I	P	IxP	Categoría	Mitigación
R1	Alteración maliciosa de expresiones vía la interfaz web	5	2	10	Medio	RNF-SEC-003: Validación Regex + Pydantic obligatoria en todas las entradas
R2	Acceso no autorizado al historial	3	3	9	Medio	RNF-SEC-001: JWT + bcrypt + HTTPS
R3	Borrado accidental de cálculos	4	4	16	Crítico	Diálogo de confirmación + borrado lógico
R4	Error por unidad DEG/RAD	4	3	12	Alto	Toggle visible con indicador prominente
R5	Crash durante cálculos complejos	3	4	12	Alto	Autosave + ProcessPoolExecutor aislado

ID	Riesgo	I	P	IxP	Categoría	Mitigación
R6	Pérdida de historial por expiración de sesión	4	2	8	Medio	Respaldos periódicos de PostgreSQL + exportación manual desde la interfaz web
R7	Incompatibilidad con versiones de navegador	3	3	9	Medio	CI/CD con matriz de navegadores (Chrome, Firefox, Safari, Edge)

3.9 Plan de Pruebas y Métricas de Calidad (ISO/IEC 25010)

3.9.1 Pruebas Unitarias (Backend – mínimo 7)

#	Prueba	Módulo	Descripción
1	test_basic_arithmetic	domain/engine	Verifica +, −, ×, ÷ con valores límite y división por cero
2	test_trig_deg_mode	domain/engine	sin(90°)=1, cos(0°)=1, tan(45°)=1 con precisión de 10 dígitos
3	test_trig_rad_mode	domain/engine	sin(π/2)=1, cos(π)=-1 con mpmath
4	test_regex_validation	application/validators	Valida que inputs inválidos son rechazados por Regex
5	test_history_crud	application/use_cases	CRUD completo de historial con borrado lógico
6	test_jwt_auth	infrastructure/security	Generación, validación y expiración de tokens JWT
7	test_session_management	application/use_cases	Creación, validación y expiración de sesiones web

3.9.2 Herramientas de Prueba

Tipo	Herramienta	Propósito
Análisis estático	SonarQube	Bugs, vulnerabilidades, code smells. Target: Rating 'A'
Pruebas E2E (Frontend)	Selenium	Automatización E2E sobre el prototipo web
Pruebas funcionales (API)	Postman (Automatizado)	Validación de 7 CRUDs + endpoints de autenticación
Pruebas de rendimiento	k6 + Lighthouse	Latencia P95, throughput, errores bajo carga, LCP
Pruebas unitarias (Backend)	pytest + pytest-cov	Cobertura > 90%
Pruebas unitarias (Frontend)	Jest	Cobertura > 80%

3.9.3 Métricas Clave a Documentar

El reporte de resultados de pruebas debe incluir: tiempos de respuesta por endpoint (P50, P95, P99); throughput (requests por segundo) bajo carga de 100, 500 y 1000 usuarios concurrentes; tasa de errores bajo carga (< 1%); cobertura de código desglosada por módulo; listado de bugs, vulnerabilidades y code smells detectados por SonarQube; áreas problemáticas identificadas según ISO/IEC 25010 (Adecuación Funcional, Eficiencia en Rendimiento, Compatibilidad, Usabilidad, Fiabilidad, Seguridad, Mantenibilidad, Portabilidad); y tareas de mejora priorizadas (refactorizaciones, correcciones de vulnerabilidades, optimización de tiempos de respuesta).

3.10 Recomendaciones de Control (COBIT)

APO12 (Gestionar el Riesgo): Formalizar el perfil de riesgo priorizando la integridad de resultados matemáticos por encima de nuevas funcionalidades. DSS05 (Gestionar Servicios de Seguridad): Implementar autenticación JWT con HTTPS, validación Regex obligatoria y manejo seguro de sesiones web. DSS04 (Gestionar Continuidad): Diseñar mecanismo de exportación de historial para prevenir pérdida de datos ante interrupciones de sesión. MEA01 (Monitorear Desempeño): Log de errores del servidor para detección proactiva de fallas y monitoreo de latencia de endpoints.